O algoritmo do EP1, passo a passo

Oct 12, 2026 · @Francisco

Introdução

O programa recebe um autómato em JSON e devolve o **AFD mínimo e completo** que aceita a mesma linguagem. Para isso aplica 7 passos, sempre pela mesma ordem (`ep1_entrega.ml:139` e `:153`):

Passo	Função	O que faz
1	`ler_automato`	lê o JSON e cria o autómato
2	`inverter`	inverte as setas e troca iniciais com finais
3	`determinizar`	transforma o resultado num AFD
4	`inverter`	inverte outra vez
5	`determinizar`	determiniza outra vez — aqui o AFD fica mínimo
6	`completar`	acrescenta um estado armadilha, se faltarem transições
7	`para_json`	escreve o resultado em JSON

Os passos 2 a 5 são o **método de Brzozowski**: inverter, determinizar, inverter, determinizar.

O exemplo: autómato M1 do enunciado

Vamos seguir o M1 (`testes/01_enunciado_M1.json`) por todos os passos.

- Estados: 0, 1, 2, 3, 4
- Alfabeto: a, b
- Estado inicial: 0
- Estados finais: 0, 2, 4

Estado	com `a` vai para	com `b` vai para
0	0	1
1	2	3
2	2	4

Estado	com `a` vai para	com `b` vai para
3	2	3
4	4	4

A biblioteca Yojson

O programa usa **só duas funções** da Yojson (versão 2.2.2): `Yojson.Basic.from_channel` para ler e `Yojson.Basic.to_string` para escrever. Todo o resto — passar de JSON para listas OCaml e voltar — é código do próprio programa. O `Makefile` liga a biblioteca com `ocamlfind ocamlopt -package yojson`.

O tipo `Yojson.Basic.t`: um valor JSON em OCaml

A Yojson representa qualquer JSON como uma árvore do tipo `Yojson.Basic.t`. Cada tipo de valor JSON tem o seu construtor (são *variantes polimórficas*, por isso começam com acento grave `` ` ``):

Valor JSON	Construtor OCaml	Exemplo	Usado no EP1?
número inteiro	`` `Int of int ``	`3` → `` `Int 3 ``	sim (estados)
texto	`` `String of string ``	`"a"` → `` `String "a" ``	sim (símbolos)
lista `[...]`	`` `List of t list ``	`[0,1]` → `` `List [`Int 0; `Int 1] ``	sim (tudo o resto)
número decimal	`` `Float of float ``	`1.5`	não
`true` / `false`	`` `Bool of bool ``	`true`	não
`null`	`` `Null ``	`null`	não
objeto `{...}`	`` `Assoc of (string * t) list ``	`{"x":1}`	não

Exemplo: a transição `[[0,"a"],1]` é lida como

```
`List [ `List [ `Int 0; `String "a" ]; `Int 1 ]
```

`Yojson.Basic.from_channel` — ler

```
val from_channel : ?buf:Buffer.t -> ?fname:string -> ?lnum:int -> in_channel
-> Yojson.Basic.t
```

Lê todo o texto de um canal de entrada e devolve a árvore `Yojson.Basic.t`. No EP1 (`ep1_entrega.ml:154`) o canal é `stdin`, por isso o programa corre com `./ep1 < testes/01_enunciado_M1.json`. Os argumentos com `?` são opcionais e não são usados.

- Espaços e mudanças de linha são ignorados: por isso o teste 19 (JSON em várias linhas) funciona.
- Se o texto não for JSON válido, lança a exceção `Yojson.Json_error`. Exemplo testado: com o input cortado a meio, o programa termina com `Fatal error: exception Yojson__Common.Json_error("Line 2, bytes -1-0: Unexpected end of input")`.

`Yojson.Basic.to_string` — escrever

```
val to_string : ?buf:Buffer.t -> ?len:int -> ?suf:string -> ?std:bool ->
Yojson.Basic.t -> string
```

Faz o contrário: recebe uma árvore e devolve o texto JSON **compacto**, numa só linha e sem espaços. Exemplo: `` `List [`Int 1; `String "a"] `` → `[1,"a"]`. No EP1 (`:155`) o texto vai para `print_endline`. Se se quisesse uma saída indentada, usava-se `Yojson.Basic.pretty_to_string`.

As funções do programa que fazem a ponte

A Yojson só trata de texto ↔ árvore. Passar da árvore para o registo `automato` (e voltar) é feito por estas funções, todas com *pattern matching* sobre os construtores acima:

Função	Recebe	Devolve	Se o formato estiver errado
`para_int` (`:34`)	`` `Int n ``	`n`	`Failure "int"`
`para_string` (`:35`)	`` `String s ``	`s`	`Failure "string"`
`para_lista` (`:36`)	`` `List l ``	`l` (lista de valores JSON)	`Failure "lista"`
`para_transicao` (`:38`)	`` `List [`List [o; s]; d] ``	`(o, s, d)`	`Failure "transicao"`
`ler_automato` (`:42`)	`` `List `` com exatamente 5 partes	registo `automato`	`Failure "automato"`

Função	Recebe	Devolve	Se o formato estiver errado
`para_json` (`:141`)	registo `automato`	`` `List `` com 5 partes	—

Exemplo testado: `[["x"],["a"],[],[0],[0]]` é JSON válido (a Yojson aceita-o), mas o estado `"x"` não é inteiro, por isso `para_int` falha com `Failure("int")`.

O fluxo completo do `main` (`:153-155`):

```
stdin ──from_channel──▶ Yojson.Basic.t ──ler_automato──▶ automato
      ──minimizar──▶ automato ──para_json──▶ Yojson.Basic.t ──to_string──▶
print_endline
```

Nota para a defesa: a Yojson já traz funções equivalentes em `Yojson.Basic.Util` (`to_int`, `to_string`, `to_list`), que lançam `Yojson.Basic.Util.Type_error` em vez de `Failure`. O programa escreveu as suas próprias versões; o comportamento é o mesmo.

Passo 1 — Ler o JSON (`ler_automato`)

O programa lê o texto JSON do *stdin* e guarda-o num registo `automato` com cinco listas (`ep1_entrega.ml:26-49`).

O JSON do M1 é uma lista de 5 partes, por esta ordem:

```
[ [0,1,2,3,4],                          ← estados
  ["a","b"],                            ← alfabeto
  [[[0,"a"],0], [[0,"b"],1], ... ],     ← transições: [[origem, símbolo],
destino]
  [0],                                  ← iniciais
  [0,2,4] ]                             ← finais
```

O que acontece a cada parte:

1. **Estados e alfabeto** são convertidos para `int list` e `string list`.
2. **Cada transição** `[[o, s], d]` passa a um triplo `(o, s, d)`. Exemplo: `[[0,"b"],1]` → `(0, "b", 1)`.
3. **Iniciais e finais** passam por `List.sort_uniq compare`, que ordena e tira repetidos. Exemplo: `[1,1,0]` → `[0;1]`.

Porque ordenar? Mais à frente, cada estado do AFD é um **conjunto de estados** guardado como lista. Se as listas estiverem sempre ordenadas e sem repetidos, o mesmo conjunto tem sempre a mesma lista, e o programa pode compará-los com `=`.

Resultado para M1:

```
estados  = [0;1;2;3;4]
alfabeto = ["a";"b"]
trans    = [(0,a,0); (0,b,1); (1,a,2); (2,a,2); (2,b,4);
            (4,a,4); (4,b,4); (1,b,3); (3,b,3); (3,a,2)]
iniciais = [0]
finais   = [0;2;4]
```

Passo 2 — Inverter (`inverter`)

`inverter` (`ep1_entrega.ml:51`) faz duas coisas:

1. **Vira cada seta ao contrário:** `(origem, símbolo, destino)` passa a `(destino, símbolo, origem)`.
2. **Troca iniciais com finais:** os finais passam a ser iniciais e o inicial passa a ser final.

O autómato invertido aceita as palavras de M1 **escritas ao contrário**. Exemplo: M1 aceita `ba` (0 —b→ 1 —a→ 2, e 2 é final), logo o invertido aceita `ab` (2 —a→ 1 —b→ 0, e 0 é final).

Para M1 invertido:

- Iniciais: **0, 2, 4** (eram os finais)
- Finais: **0** (era o inicial)

Transição em M1	Transição invertida
0 —a→ 0	0 —a→ 0
0 —b→ 1	1 —b→ 0
1 —a→ 2	2 —a→ 1
2 —a→ 2	2 —a→ 2
2 —b→ 4	4 —b→ 2
4 —a→ 4	4 —a→ 4
4 —b→ 4	4 —b→ 4
1 —b→ 3	3 —b→ 1
3 —b→ 3	3 —b→ 3
3 —a→ 2	2 —a→ 3

Repara que o resultado **já não é determinista**: o estado 2 com `a` vai para 1, 2 e 3; e há três estados iniciais. Por isso o passo seguinte é determinizar.

Resumo do invertido, por estado e símbolo (vai ser usado no passo 3):

Estado	com `a` vai para	com `b` vai para
0	{0}	{}
1	{}	{0}
2	{1, 2, 3}	{}
3	{}	{1, 3}
4	{4}	{2, 4}

Passo 3 — Determinizar (`determinizar`)

`determinizar` (`ep1_entrega.ml:66`) é a **construção por subconjuntos**: cada estado do novo AFD é um *conjunto* de estados do autómato anterior.

As peças do código

- **`mover trans conj simb`** (`:57`): dado um conjunto e um símbolo, junta todos os destinos possíveis e devolve-os ordenados e sem repetidos. Exemplo: `mover _ [0;2;4] "a"` = {0} ∪ {1,2,3} ∪ {4} = `[0;1;2;3;4]`.
- **`registados`**: lista de pares *(conjunto, número)*. Guarda os conjuntos já encontrados e o número que cada um recebeu. Um conjunto novo recebe o número seguinte (`List.length registados`).
- **`por_tratar`**: a fila de conjuntos que ainda falta processar. Os novos entram no **fim** (`resto @ descobertos`), por isso é uma pesquisa em largura (BFS).
- **`tratar`** (`:72`): para um conjunto, percorre o alfabeto; para cada símbolo chama `mover`, regista o destino se for novo e cria a transição.
- **Conjunto vazio:** se `mover` devolve `[]`, não é criada transição (`:76`). O AFD pode ficar com transições em falta; isso só é resolvido no passo 6.
- **Finais:** um conjunto é final se contiver pelo menos um estado final do autómato de entrada (`eh_final`, `:71`).

Aplicação ao M1 invertido

Início: o conjunto inicial é a lista de iniciais **{0, 2, 4}**, que recebe o número **0**. `registados = [({0,2,4}, 0)]`, `por_tratar = [{0,2,4}]`.

Iteração	Conjunto tratado	com `a` (mover)	com `b` (mover)	Novos registados	Fila depois
1	{0,2,4} = **0**	{0}∪{1,2,3}∪{4} = {0,1,2,3,4} → novo **1**	{}∪{}∪{2,4} = {2,4} → novo **2**	1, 2	[1, 2]
2	{0,1,2,3,4} = **1**	{0}∪{1,2,3}∪{4} = {0,1,2,3,4} → já é 1	{0}∪{1,3}∪{2,4} = {0,1,2,3,4} → já é 1	—	[2]
3	{2,4} = **2**	{1,2,3}∪{4} = {1,2,3,4} → novo **3**	{}∪{2,4} = {2,4} → já é 2	3	[3]
4	{1,2,3,4} = **3**	{1,2,3,4} → já é 3	{0}∪{1,3}∪{2,4} = {0,1,2,3,4} → já é 1	—	[] → fim

Finais: o único final do invertido é o estado 0. Os conjuntos que contêm 0 são {0,2,4} e {0,1,2,3,4}, logo os finais são **0 e 1**.

Resultado do passo 3 (chamemos-lhe D1)

Os estados são renumerados 0, 1, 2, 3. Inicial: 0. Finais: 0, 1.

Estado	Conjunto original	com `a`	com `b`	Final?
0	{0,2,4}	1	2	sim
1	{0,1,2,3,4}	1	1	sim
2	{2,4}	3	2	não
3	{1,2,3,4}	3	1	não

D1 é um AFD (uma só seta por estado e símbolo) que aceita as palavras de M1 ao contrário. Todos os seus estados são alcançáveis a partir do inicial, porque a BFS só cria estados a que consegue chegar. Estas duas propriedades são as que fazem o passo 5 dar um AFD mínimo.

Passo 4 — Inverter outra vez (`inverter`)

A mesma função do passo 2, agora aplicada a D1. Como as palavras são invertidas duas vezes, este autómato volta a aceitar **as palavras originais de M1**.

- Iniciais: **0, 1** (eram os finais de D1)
- Finais: **0** (era o inicial de D1)

Transição em D1	Transição invertida
0 —a→ 1	1 —a→ 0
0 —b→ 2	2 —b→ 0
1 —a→ 1	1 —a→ 1
1 —b→ 1	1 —b→ 1
2 —a→ 3	3 —a→ 2
2 —b→ 2	2 —b→ 2
3 —a→ 3	3 —a→ 3
3 —b→ 1	1 —b→ 3

Resumo por estado e símbolo (usado no passo 5):

Estado	com `a` vai para	com `b` vai para
0	{}	{}
1	{0, 1}	{1, 3}
2	{}	{0, 2}
3	{2, 3}	{}

De novo não é determinista (dois iniciais; o estado 1 com `a` vai para 0 e 1), por isso segue-se outra determinização.

Passo 5 — Determinizar outra vez (`determinizar`)

A mesma função do passo 3, com o mesmo procedimento. A diferença é o resultado: **este AFD já é o mínimo**.

Início: conjunto inicial **{0, 1}** = número **0**. `por_tratar = [{0,1}]`.

Iteração	Conjunto tratado	com `a` (mover)	com `b` (mover)	Novos registados	Fila depois
1	{0,1} = **0**	{}∪{0,1} = {0,1} → já é 0	{}∪{1,3} = {1,3} → novo **1**	1	[1]
2	{1,3} = **1**	{0,1}∪{2,3} = {0,1,2,3} → novo **2**	{1,3}∪{} = {1,3} → já é 1	2	[2]

Iteração	Conjunto tratado	com `a` (mover)	com `b` (mover)	Novos registados	Fila depois
3	{0,1,2,3} = **2**	{}∪{0,1}∪{}∪{2,3} = {0,1,2,3} → já é 2	{}∪{1,3}∪{0,2}∪{} = {0,1,2,3} → já é 2	—	[] → fim

Finais: o final é o estado 0. Contêm 0 os conjuntos {0,1} e {0,1,2,3}, logo os finais são **0 e 2**.

Resultado do passo 5

Estado	com `a`	com `b`	Final?	O que significa
0 (inicial)	0	1	sim	até agora só li `a`'s
1	2	1	não	li `a`'s seguidos de um ou mais `b`'s
2	2	2	sim	já apareceu um `a` depois de um `b`

M1 tinha 5 estados e o resultado tem 3. Em M1 os estados 1 e 3 faziam o mesmo trabalho, tal como 2 e 4; a dupla inversão juntou-os.

Porque é que fica mínimo

É o **teorema de Brzozowski**: se D é um AFD em que todos os estados são alcançáveis, então determinizar o inverso de D dá sempre o AFD mínimo.

O passo 3 garantiu exatamente isso para D1 (é determinista e a BFS só criou estados alcançáveis). Os passos 4 e 5 são "determinizar o inverso de D1", por isso o resultado é mínimo.

Intuição: no passo 5, cada estado é o conjunto dos estados de D1 a partir dos quais ainda se pode aceitar. Dois estados que aceitariam exatamente as mesmas continuações dão origem ao mesmo conjunto, logo nunca ficam separados.

Porque é preciso o passo 3 antes? Porque o teorema só vale se o autómato a inverter for determinista e só tiver estados alcançáveis. A entrada pode ser um AFND ou ter estados inalcançáveis; o passo 3 resolve as duas coisas.

Passo 6 — Completar (`completar`)

Um AFD completo tem, para cada estado, uma transição com cada símbolo. Como o passo 5 nunca cria o conjunto vazio, podem faltar transições. `completar` (`ep1_entrega.ml:112`) trata disso em três casos:

1. **Sem estados finais** (linguagem vazia): devolve logo um AFD de 1 estado, não final, com laços em todos os símbolos.

2. **Já completo**: se o número de transições for igual a *estados × símbolos*, não muda nada. Chega contar porque, depois de determinizar, nunca há duas transições com o mesmo estado e símbolo.
3. **Faltam transições**: cria um estado armadilha com o número *n* (o primeiro livre), não final. Cada transição em falta passa a ir para a armadilha, e a armadilha fica com um laço em cada símbolo.

No M1

Há 3 estados × 2 símbolos = 6 transições esperadas, e o passo 5 produziu 6. É o caso 2: **nada muda**.

Num exemplo em que falta algo (`testes/06_afd_parcial_armadilha.json`)

Este autómato só aceita a palavra `ab`. Depois do passo 5 fica assim: 0 —a→ 1, 1 —b→ 2, final 2. São 2 transições, mas deviam ser 3 × 2 = 6.

O que `completar` acrescenta	Porquê
Estado 3 (armadilha)	3 é o primeiro número livre
0 —b→ 3	faltava (0, b)
1 —a→ 3	faltava (1, a)
2 —a→ 3 e 2 —b→ 3	faltavam (2, a) e (2, b)
3 —a→ 3 e 3 —b→ 3	laços da armadilha

A armadilha não é final, por isso a linguagem não muda. O AFD continua mínimo: o resultado do passo 5 não tinha nenhum estado "morto", por isso a armadilha é o único e não fica repetida.

Passo 7 — Escrever o JSON (`para_json`)

`para_json` (`ep1_entrega.ml:141`) faz o inverso do passo 1: volta a pegar no registo e escreve-o no mesmo formato de 5 partes. Cada triplo `(o, s, d)` volta a ser `[[o, s], d]`. O `main` (`:153`) imprime o resultado numa linha.

Saída real do programa para M1:

```
[[0,1,2],["a","b"],[[[0,"a"],0],[[0,"b"],1],[[1,"a"],2],[[1,"b"],1],
[[2,"a"],2],[[2,"b"],2]],[0],[0,2]]
```

- Estados: 0, 1, 2

- Transições: as 6 da tabela do passo 5
- Inicial: 0 (é sempre 0, porque o conjunto inicial é o primeiro a ser registado)
- Finais: 0, 2

O caminho completo de M1

Passo	Autómato	Estados	Determinista?
Entrada	M1	5	sim
2 — inverter	M1 invertido	5 (3 iniciais)	não
3 — determinizar	D1	4	sim
4 — inverter	D1 invertido	4 (2 iniciais)	não
5 — determinizar	AFD mínimo	3	sim
6 — completar	AFD mínimo completo	3	sim